

ANOGS / AnoSDK

Anti-Cheat Runtime

Deep Reverse-Engineering & Anti-Tamper Resilience Assessment
of a stripped ARM64 iOS anti-cheat dynamic library

SCOPE: DEFENSIVE HARDENING ONLY — NO BYPASS CONTENT

TARGET	anogs45orgnll
FILE TYPE	Mach-O 64-bit dylib · ARM64 (ARM64_ALL)
SIZE	2,871,200 bytes
SHA-256	0a9943f19c78194e377210dce5b349cf13b4e544dda63668cb39239381b159c1
MD5	c2c2ca25a5097afb75046cf10a4f2572
TOOLCHAIN	LIEF 1.0 · Capstone 5.0 · llvm-objdump · indirect-symtab
METHOD	Static disassembly · dual-disassembler cross-check
DATE	2026-08-26

This document identifies structural weak points where the anti-cheat could lose detection or evidence, and prescribes defensive hardening. It contains no patch bytes, hooks, NOP/RET edits, or bypass procedures. Offsets are provided so the developer can independently re-verify every finding.

§ Table of Contents

1	Executive Summary	14	Call-Graph Analysis
2	Binary Overview	15	Return-Value Analysis
3	Anti-Cheat Architecture	16	Fail-Open Analysis
4	Initialization	17	Critical Security Offsets
5	Security Context & Global State	18	Single Points of Failure
6	Collectors	19	Attack-Surface Matrix
7	Detectors	20	Defensive Hardening Plan
8	Integrity Engine	21	Anti-Cheat Master Map
9	Event System	22	Critical Dependency Map
10	Dispatcher	23	Failure Propagation Map
11	Aggregator	24	Top 10 Security Weak Points
12	Reporting Pipeline	25	Final Technical Conclusions
13	Transport		

Confidence: **CONFIRMED** direct from disassembly **HIGH** multiple strong signals **INFERENCE** not fully proven | Severity: **CRITICAL** **HIGH** **MEDIUM** **LOW**

1 Executive Summary

Target. `anogs45orgnll` is a stripped 64-bit ARM Mach-O **dynamic library** (2.74 MB) that implements a commercial mobile anti-cheat runtime. The exported API is prefixed `AnoSDK*` and the internal telemetry uses tags such as `COREREPORT`, `tdm_report`, `TSS`, `ms_*` and `mrpcs_*`. This identifies the module as the **ANOGS / AnoSDK** family (Ano Game Security). **INFERENCE** for the exact product name; **CONFIRMED** that it is an anti-cheat/anti-tamper SDK from its exports, imports and strings.

What it does. At load time 59 initializers register a set of collectors and detectors (anti-debug, anti-injection, memory/code integrity, Objective-C runtime tamper, jailbreak/emulator, device fingerprint). Detections are converted into **named events** that are aggregated and pushed through a single logical report channel (`COREREPORT`) to a network transport. A large part of the sensitive detection logic is protected by **control-flow flattening** and a **custom bytecode VM** (`vm_main.img` / `vm_x_task.img`).

Headline defensive findings. The obfuscation makes the module hard to *tamper with*, but the *result-handling* and *reporting* paths contain several **fail-open** and **silent-drop** constructs where a benign failure (a blocked syscall, an uninitialised channel, a null object) is treated as "clean" or causes an event to be discarded with no error propagation. In an anti-cheat, losing a detection is as damaging as never running it. The most important structural risks are:

#	Weak point	Offset (VA)	Effect if it fails	Sev.	Conf.
1	Single report channel gate (<code>COREREPORT</code> availability flag)	0x29bc38 · sub_0371e0 / sub_0373e8	All detector evidence is silently dropped process-wide	CRITICAL	CONFIRMED
2	Report emit has 5 null-guards that return without emitting	sub_0373e8 @ 0xf373e8	Event created but never dispatched (Event Drop)	CRITICAL	CONFIRMED
3	Scan/integrity event classifier is a single fan-in = 1 node	sub_102d48	All <code>ms_</code> / <code>mrpcs_</code> integrity events lose classification	CRITICAL	HIGH
4		sub_037004 @ 0x37004		HIGH	CONFIRMED

	Debugger check returns "clean" when <code>sysctl</code> itself fails		Anti-debug blinded by blocking one syscall (Fail-Open)		
5	<code>AnoSDKInit</code> return value is derived from input, not init success	<code>sub_0f0fd0</code> / <code>sub_02d69c</code>	Host cannot tell if protection actually came up	HIGH	CONFIRMED

Bottom line. The binary's anti-tamper surface (obfuscation, VM, raw syscalls, redundant collectors) is strong. Its **weakest links are logical, not cryptographic**: centralised evidence routing with client-trusted fail-open behaviour. Hardening should focus on making the detect→event→report path **fail-closed**, redundant, and server-verified, so that suppressing one function or syscall degrades detection *observably* rather than silently. Sections 15–24 quantify and localise each issue with re-verifiable offsets; Section 20 gives the remediation plan.

2 Binary Overview

2.1 Identification CONFIRMED

Format	Mach-O 64-bit, <code>MH_DYLIB</code> (dynamically-linked shared library)
Architecture	ARM64, <code>cpu_subtype 0 = CPU_SUBTYPE_ARM64_ALL</code> → plain ARM64, not ARM64e (no pointer authentication / PAC)
Header flags	<code>NOUNDEFS · DYLDLINK · TWOLEVEL · BINDS_TO_WEAK · NO_REEXPORTED_DYLIBS</code> ; 28 load commands
Symbol state	Stripped of internal names: only 19 named symbols fall inside <code>__text</code> (the <code>AnoSDK*</code> exports). All internal routines are referenced here as <code>sub_<VA></code> .
Exports	21 (19 C functions + Obj-C class <code>PluginTssSDKLifecycle</code> and its metaclass)
Imports	369 external symbols from libSystem, ObjC, Foundation, UIKit, Security, SystemConfiguration, CFNetwork
Recovered functions	10,121 (from exports + initializers + 36,348 internal call targets)

2.2 Segments & the offset model CONFIRMED

The image base is `0` . For the two mapped segments that hold code and data, the virtual address equals the file offset, so **within `__TEXT` and `__DATA` : VA = RVA = File-Offset**. Only `__LINKEDIT` diverges (by `0x14000` , because `__DATA` 's zero-fill virtual size exceeds its on-disk size). **Every offset in this report is a VA**, which for all code/data referenced here is numerically identical to the file offset. This eliminates VA/RVA/file-offset ambiguity for re-verification.

Segment	VA start	V.size	File off	F.size	init/max prot	VA==FileOff?
<code>__TEXT</code>	<code>0x00000000</code>	<code>0x283000</code>	<code>0x000000</code>	<code>0x284000</code>	r-x / r-x	yes ($\Delta=0$)
<code>__DATA</code>	<code>0x00284000</code>	<code>0x02c000</code>	<code>0x284000</code>	<code>0x018000</code>	rw- / rw-	yes ($\Delta=0$)
<code>__LINKEDIT</code>	<code>0x002b0000</code>	<code>0x024000</code>	<code>0x29c000</code>	<code>0x020fe0</code>	r-- / r--	no ($\Delta=0x14000$)

Key sections

Section	VA	Size	Role in analysis
<code>__text</code>	<code>0x004190</code>	<code>0x235508</code>	All executable code (2.32 MB)
<code>__stubs</code>	<code>0x23a318</code>	<code>0x0d8c</code>	289 import stubs (all resolved via indirect symtab)
<code>__const</code> / <code>__cstring</code>	<code>0x23be10</code>	—	Jump tables, C strings, event tags
<code>__mod_init_func</code>	<code>0x284b78</code>	<code>0x1d8</code>	59 load-time initializers
<code>__objc_classlist</code>	<code>0x28beb0</code>	<code>0x30</code>	Own Obj-C classes (incl. <code>PluginTssSDKLifecycle</code>)
<code>__data</code> / <code>__bss</code> / <code>__common</code>	<code>0x28dbe0+</code>	—	Global security state (flags, counters, singletons)

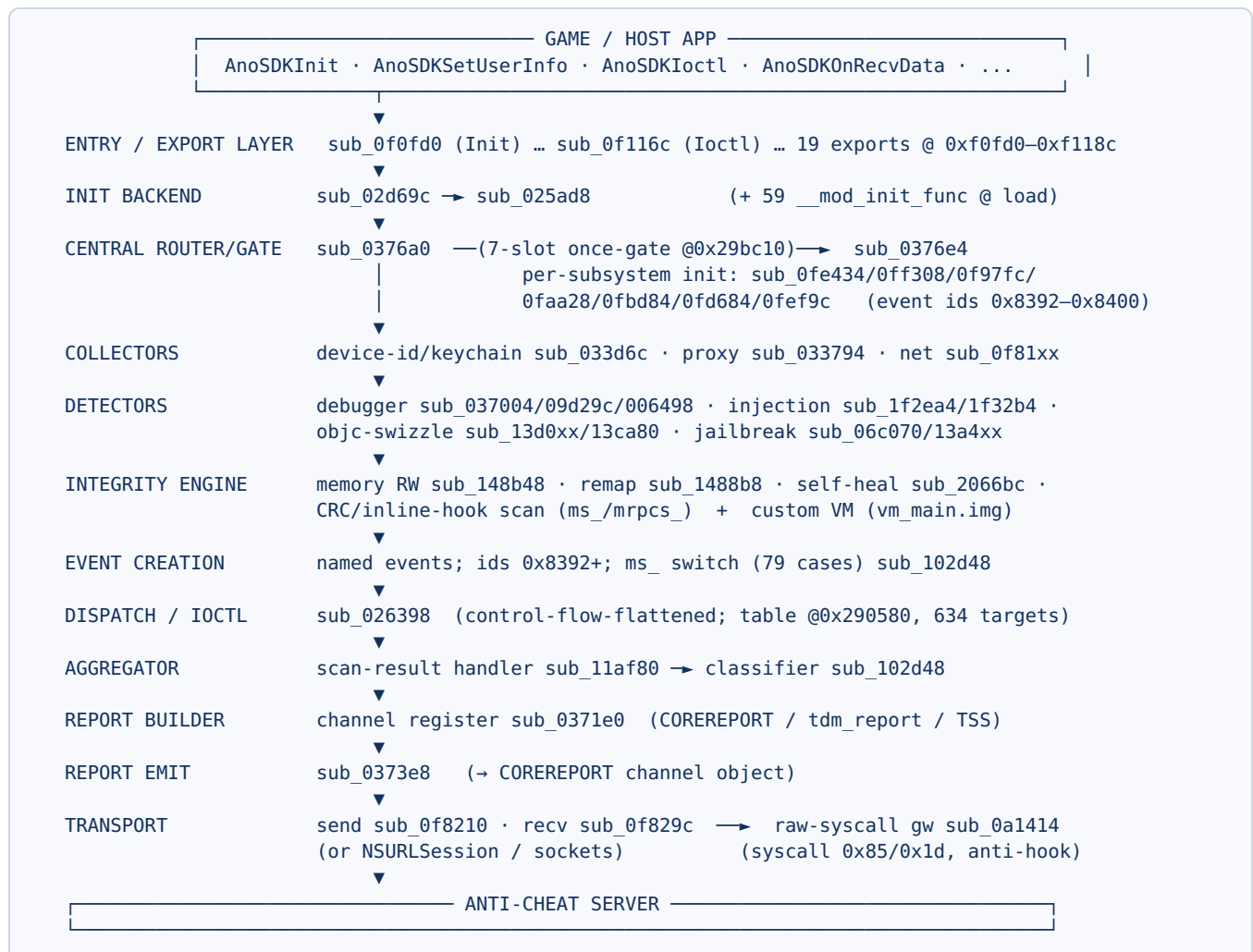
Anti-analysis note. The primary command router `sub_026398` is **control-flow-flattened** and dispatches through an 8-byte successor table at `0x290580` spanning **634** in-`__text` targets. A linear-sweep or naive recursive-descent call graph therefore **under-counts** edges out of this and other VM-protected functions. Findings that depend on those hidden edges are labelled **HIGH** / **INFERENCE** accordingly.

2.3 ASLR assumptions

As a dylib the module is fully position-independent and is slid by dyld at load. All internal control transfers observed are PC-relative (`adrp/add`, `bl`, `ldr` literal), and imports resolve through the stub/GOT tables — so the slide affects absolute addresses uniformly and does **not** change any of the VAs used here relative to the image base. Any runtime self-check that hard-codes absolute addresses would need `_dyld_get_image_vmaddr_slide`; that import is present and used (see §7.2), which is consistent with slide-aware self-integrity checks. **CONFIRMED** import; **HIGH** on usage intent.

3 Anti-Cheat Architecture

Reconstructed from exports, the 59 initializers, import cross-references and telemetry strings. Stage names are functional; the concrete backing functions/offsets are given (guessed labels are marked).



Solid backbone confirmed by disassembly (§4, §10, §12, §13). The Collector/Detector/Integrity boxes are populated from import XREFs (§7) and are individually attributed with confidence levels; a detector's placement in a "stage" is functional grouping, not a claim of a literal linear pipeline.

4 Initialization

4.1 AnoSDKInit → backend CONFIRMED

AnoSDKInit (0xf0fd0) calls the init backend **sub_02d69c**, then computes its own return value from the input argument and tail-calls the central router with command id 0:

```
0xf0fd0 AnoSDKInit:
    mov x19, x0                ; save caller argument
    bl sub_02d69c              ; init backend (return value in x0 is DISCARDED)
    mvn w8, w19                ; w8 = ~arg
    lsr w0, w8, #0x1f          ; w0 = (arg >= 0) ? 1 : 0 ← return derived from INPUT
    mov w1, #0
    b sub_0376a0                ; tail-call central router( w0 , cmd=0 )

0x02d69c init_backend:
    ... build {0x10, arg} on stack ...
    bl sub_025ad8              ; real init work (getpid/kill-based liveness, telemetry)
    mov w0, #0                 ; ← ALWAYS returns 0, regardless of sub_025ad8 result
    ret
```

Finding 4.1 (Return-value integrity). Neither the backend result nor any subsystem-init result reaches the value the host receives from **AnoSDKInit**. **sub_02d69c** hard-returns 0, and **AnoSDKInit** returns a boolean computed only from the *sign of its argument*. A host that checks **AnoSDKInit()** to decide "is protection up?" is checking argument validity, not protection health. HIGH · CONFIRMED.

4.2 Load-time initializers

59 pointers in **__mod_init_func** (0x284b78) run before the host calls anything — e.g. 0x1013c, 0x49ee4, 0x94318, 0xd2ed0, 0x1d497c...0x208fac. These construct the C++ subsystem singletons and the VM. Because they run at **dlopen** time, the protection's own bring-up is **not** gated by **AnoSDKInit**'s return, reinforcing Finding 4.1: the host has no single success signal for the ~59-initializer bring-up.

4.3 The central router / once-gate **sub_0376a0** CONFIRMED

Every export that "starts" a subsystem funnels through **sub_0376a0(value, cmd)**. It first runs a throttled trigger **sub_0370d0** (a **time()**-based ~10% sampler that fires **sub_0effc8** + **sub_0f0914** once), then indexes a **7-entry once-only table** at 0x29bc10 by **cmd**:

```
0x0376a0 central_router( x0=value, x1=cmd ):
    bl sub_0370d0                ; throttled one-time trigger (flag @0x29bbf8)
    cmp w20, #6                  ; cmd bound check (0..6)
    b.hi return
    adrp x9, 0x29b000 ; add x9,#0xc10 ; table @0x29bc10
    ldr w8, [x9, w20, uxtw #2]    ; slot = already_ran[cmd]
    cbz w8, 0x376e4              ; if 0 → run subsystem once ; else return
0x0376e4:
    mov w10, #1 ; str w10, [x9, x8, lsl #2] ; mark already_ran[cmd]=1
    ldrb w11, [0x23c6d0 + cmd] ; br → per-cmd init: ; byte jump-table
    cmd0→sub_0fe434(0x8392) cmd1→sub_0ff308(0x839f) cmd2→sub_0f97fc(0x83b3)
    cmd3→sub_0faa28(0x83c3) cmd4→sub_0fbd84(0x83d4) cmd5→sub_0fd684(0x83ea)
    cmd6→sub_0fef9c(0x8400) ; (cmd4/5/6 also capped ≤3 via 0x29bc04/08/0c)
    ; --- result handling ---
    mov x20, x0
    cbz x0, return                ; (A) subsystem init returned 0 → SILENT skip
    bl sub_017518 ; cbz x0, return ; (B) field build failed → SILENT skip
    adr x0, "monitor" ; bl sub_0371e0(..., 1) ; str [sp, #8]
    cbz x0, return                ; (C) monitor/report register failed → SILENT skip
    ldr x8, [x0]; ldr x8, [x8, #0x28]; blr x8 ; start monitor (virtual call)
    ... "%d" → sub_0373e8 into global list @0x291960
```

Finding 4.3 (Silent subsystem skip). Three consecutive `cbz x0, return` guards (A/B/C) mean that if a detector subsystem fails to construct, fails to build its report field, or fails to register with the monitor, **it is silently skipped** and the router returns as if nothing was wrong — no error, no fallback, no telemetry that a subsystem is missing. Combined with §4.1 the host cannot observe partial bring-up. **HIGH** · **CONFIRMED**.

5 Security Context & Global State

The protection keeps its live state in a cluster of `__DATA` globals around `0x29b000` and `0x2ab000`. These small words are the *de-facto* security context; several are the exact toggles that decide whether detection or reporting happens.

Global (VA)	Meaning (from XREF)	Security relevance	Conf.
0x29bbf8	"router bootstrap done" flag (<code>sub_0370d0</code>)	One-shot; gates the throttled trigger	CONFIRMED
0x29bc10[7]	"subsystem already-ran" table (cmd 0..6)	Once-only init; a stray write disables a subsystem's start	CONFIRMED
0x29bc04/08/0c	Per-cmd run counters (cap 3)	Rate-limit for cmd4/5/6 subsystems	CONFIRMED
0x29bc38	Report channel AVAILABLE flag (<code>COREREPORT</code>)	Master gate for all telemetry (§12)	CONFIRMED
0x29bc39	Report registration ATTEMPTED flag	Prevents re-registration	CONFIRMED
0x29bc3c	Report registration FAILURE counter (≥3 → disable)	Fail-open cap: gives up reporting after 3 tries	CONFIRMED
0x291960	Global report/monitor list head	Aggregation anchor for emitted records	HIGH
0x2ab430	Transport mode bitfield (bit13 send / bit14 recv)	Selects raw-syscall vs libc path (§13)	CONFIRMED
0x2ab448 / 0x2ab450	Post-send / post-recv callback pointers	Optional transport interposers	CONFIRMED

Observation. All of the above are **plain, unauthenticated words in writable `__DATA`**. There is no evidence of a checksum or shadow copy protecting the context itself. Because `0x29bc38` alone can silence every report (§12), the security context is a higher-value target than any single detector. See hardening H-2/H-7.

6 Collectors

Collectors gather environment facts that later become report fields or feed detectors.

6.1 Device identity / keychain persistence `sub_033d6c` **CONFIRMED**

`sub_033d6c` (fan-out 26) is the fingerprint hub: it calls `_CFUUIDCreate` / `_CFUUIDCreateString` to mint an id and persists it in the keychain via `_SecItemAdd` / `_SecItemCopyMatching` / `_SecItemDelete` under a `kSecClassGenericPassword` item (accessible `AfterFirstUnlock`). The "DeviceID" tag is built in `sub_0332c4` / `sub_036a9c`. This yields a persistent per-device identifier that survives reinstall.

6.2 Network environment / MITM probe `sub_033794` **CONFIRMED**

`sub_033794` calls `_CFNetworkCopySystemProxySettings` — i.e. it inspects whether an HTTP/S proxy is configured, a classic man-in-the-middle / traffic-interception signal. Reachability is monitored via the `SCNetworkReachability*` family.

6.3 Other collectors

- **Process/thread:** `_getpid` (9 callers), `pthread_*` naming/enumeration, `_sysctl` KERN_PROC.

- **Time base (anti-debug timing):** `_mach_absolute_time`, `_mach_timebase_info`, `_clock_gettime` — timing deltas around checks.
- **App/bundle facts:** `NSBundle`, `NSProcessInfo`, `UIDevice`, `NSHomeDirectory`.

7 Detectors

Each row was reached by resolving the security-relevant import to its calling function(s) (import XREF), then reading the call site — **not** from strings alone.

7.1 Anti-debug CONFIRMED

Function	Technique	Evidence
sub_037004	<code>sysctl(KERN_PROC,PID)</code> → test <code>p_flag & P_TRACED(0x800)</code>	<code>bl_getpid; bl_sysctl; and w9,w9,#0x800</code>
sub_09d29c	two <code>sysctl</code> calls (proc list / flags)	<code>2x_sysctl</code> + malloc/free buffer
sub_006498	<code>task_info</code> debug/exception state + <code>dlopen/dlsym</code> probe of debug libs	<code>_task_info, _dlopen, _dlsym, _getenv, _basename</code>
sub_0f73dc	builds the <code>"debugger"/"debugger=%s"</code> event	string XREF + 16 callees

Note: there is **no** `ptrace` import; `_syscall` is imported and used (`sub_214de8`, `sub_21d880`) so a `PT_DENY_ATTACH`-style call can be issued without a resolvable symbol — consistent with anti-hook design. See §15 for the fail-open in `sub_037004`.

7.2 Anti-injection / dynamic-loader CONFIRMED

Function	Technique	Callers
sub_1f2ea4	Enumerate loaded images: <code>_dyld_image_count</code> + <code>_get_image_name</code> + <code>_get_image_header</code>	4 (central)
sub_1f32b4	Map memory regions to files: <code>mach_vm_region_recurse</code> + <code>proc_regionfilename</code> + <code>pid_for_task</code>	5 (central)
sub_2034ec	Region walk + <code>dladdr</code> attribution of code addresses	1
sub_038f3c	Broad <code>dlopen/dlsym/dlclose/ioctl/select/kill</code> hub (see §10, §18)	super-hub

Detects injected dylibs (Substrate/Frida-style), unexpected images, and code executing from unbacked/anonymous regions. The `__VM_ASSERT` / `vm_*.img` logic (§8.3) cross-checks these results inside the VM.

7.3 Objective-C runtime tamper (swizzling) CONFIRMED

Function	Technique
sub_13d050 / sub_13d194	Resolve a method's current <code>IMP</code> via <code>class_getInstanceMethod</code> / <code>class_getClassMethod</code> + <code>NSSelectorFromString</code> and compare
sub_13ca80 / sub_13cdac	<code>method_setImplementation</code> — install/restore its own IMPs
sub_13d45c	<code>objc_allocateClassPair</code> + <code>objc_registerClassPair</code> — dynamic guard class
sub_1eb5a0 / sub_1eb758	Runtime install path for the above (initializer-driven)

7.4 Jailbreak / emulator / environment CONFIRMED

Function	Technique / string
sub_06c070	

	<code>getmntinfo</code> — detect remounted / writable root filesystem
<code>sub_13a724</code>	<code>access</code> + <code>statfs</code> on suspect paths
<code>sub_13a42c</code>	<code>stat</code> + <code>lstat</code> + <code>readlink</code> symlink checks
<code>sub_0b43b4</code>	alert: "Your device has been jailbroken, which poses a security risk..."
<code>sub_0063d4</code>	<code>root_alert</code> / <code>root_alert:%s</code> event
<code>sub_0d06b8</code> / <code>sub_0eb9a4</code>	"running in an Android emulator..." — shared cross-platform SDK strings

The Android-emulator strings indicate a shared cross-platform code/string base; on this iOS build those specific branches may be inert. Treated as **INFERENCE** for iOS-runtime relevance.

8 Integrity Engine

8.1 Memory / code integrity primitives **CONFIRMED**

Function	Primitive	Purpose (inferred)
<code>sub_148b48</code>	<code>vm_region_64</code> + <code>vm_protect</code> + <code>vm_write</code>	Read/verify/repair protection of code pages (5 callers — central)
<code>sub_1488b8</code>	<code>vm_copy</code> + <code>mprotect</code> + <code>vm_remap</code>	Remap/relocate protected regions
<code>sub_2066bc</code>	<code>vm_copy</code> + <code>mach_vm_remap</code> + <code>sys_icache_invalidate</code>	Self-heal code + flush i-cache after re-check
<code>sub_099cb8</code> / <code>sub_128930</code>	<code>mprotect</code>	Toggle W/X on scan targets

8.2 CRC / inline-hook scanner (`ms_` / `mrpcs_`) **HIGH**

The scan subsystem produces these named events (string XREF into the classifier `sub_102d48`):

Event tag	Meaning
<code>ms_data_crc</code> / <code>mrpcs_data_crc_error</code>	CRC mismatch over a protected data/code range
<code>inline_hook_opcode_mismatch</code>	First opcodes of a function differ from expected → inline hook
<code>ms_hook_opcode</code> / <code>ms_set_inlie_hook</code> / <code>set_inline_hook_error</code>	Hook-opcode state / self-hook install / install failure
<code>ms_scan_start</code> / <code>mrpcs_scan_thread_start_failed!</code>	Scan lifecycle — incl. an explicit "scan thread failed to start"

Self-reported blind-spot. `mrpcs_scan_thread_start_failed!` is a first-class event: the code *knows* the scan thread can fail to start. If that failure path only logs and does not force a server-side "integrity-unknown" verdict, a suppressed scan thread yields **integrity blindness** that looks identical to "all clean". See H-1/H-4.

8.3 Custom protection VM **HIGH**

Strings `vm_main.img`, `vm_x_task.img` (`sub_00ece4`), `[VM][%s][%s]...VM_MemPool::~~VM_MemPool, Leak cnt:%zd` (`sub_0b4680`) and `__VM_ASSERT err! %s, ln:%d` (`sub_038f3c`) establish a **custom bytecode virtual machine** that executes protected detection logic from embedded images. This is the reason the ioctl router and parts of the integrity engine are control-flow-flattened: the sensitive comparisons run as VM bytecode, not native code, so static recovery of those exact predicates is limited. The presence of the VM is **CONFIRMED** (strings + memory-pool management); the precise bytecode semantics are **out of static scope**.

9 Event System

9.1 Subsystem event-id space CONFIRMED

The router dispatch (§4.3) tags each subsystem init with a 16-bit event id passed in `w0` before the subsystem call:

Cmd	Event id	Subsystem init	Gate
0	0x8392	sub_0fe434	once
1	0x839f	sub_0ff308	once
2	0x83b3	sub_0f97fc	once
3	0x83c3	sub_0faa28	once
4	0x83d4	sub_0fbd84	once + counter 0x29bc04 (≤ 3)
5	0x83ea	sub_0fd684	once + counter 0x29bc08 (≤ 3)
6	0x8400	sub_0fef9c	once + counter 0x29bc0c (≤ 3)

Reporting-side event ids observed in `sub_0371e0`: `0x1ae14`, `0x1ae15`, `0x1ae16`, `0x18704`, `0x9c44` (field/section ids under the `TSS / COREREPORT` record). These are the record's structural tags, distinct from detector event ids.

CONFIRMED values; INFERENCE on exact field semantics.

9.2 Scan-event classifier switch (79 cases) HIGH

`sub_102d48` resolves an event-type integer via `sub_11ac6c` and dispatches through a jump table at `0x2656fc` bounded at `0x4e` (so up to **79** distinct scan event types), each building a named `ms_/mrpcs_` record with `memset/strncpy/snprintf`. It is preceded by a precondition `sub_10298c`; if that returns false the whole classification is skipped (see §16).

9.3 Reserved / unused

The router table is bounded to 7 (`cmp #6; b.hi`); ids beyond 6 are rejected (reserved). Within the 79-case scan switch, any index whose table slot points to the common `default` block is effectively reserved/unused; enumerating each requires VM-context and is left as re-verification work for the maintainer.

10 Dispatcher

10.1 `AnoSDKIoctl` → flattened backend `sub_026398` CONFIRMED

`AnoSDKIoctl` (`0xf116c`) is a bare tail-jump to `sub_026398`, the primary host ↔ SDK command interface. It is **control-flow-flattened**: a state variable (seeded `0x31c04ca0`) selects the next block through an 8-byte successor table at `0x290580`; each block mutates the state via `add/eor/csel` and does `ldr x9, [x28, w9, uxtw#3]; br x9`.

```
0x026398 ioctl_backend( w0=cmd, x1=arg ):
... stack canary from __stack_chk_guard ...           ; anti-corruption
mov  w8, #0x31c04ca0                                ; CFF state seed
...
adrp x28, 0x290580 ; ldr x9, [x28, w9, uxtw#3] ; br x9  ; flattened dispatch (634 targets)
...
"|emulator_name=" , "|%d" → builds report fields
```

Analysis limitation (honest scope). Because dispatch is data-driven through `0x290580`, the concrete `cmd→handler` mapping is **not** recoverable by static linear sweep and is partly executed as VM bytecode. The command surface is therefore **large and opaque** — a property that helps against tampering but also means any logic bug inside it (e.g. an unhandled `cmd` falling through to a benign default) cannot be ruled out statically. Recommend the maintainer audit the `cmd` table with source/symbolic execution. **MEDIUM** · **CONFIRMED (structure)**.

10.2 The super-hub `sub_038f3c` **CONFIRMED**

`sub_038f3c` is the single call site of `_ioctl` and also reaches `dlopen/dlsym/dlclose/dladdr/select/kill/getenv/symlink`. It behaves as a general OS-services broker for the detectors. Its centrality makes it both a resilience asset (one place to harden) and a single dependency (one place to lose many capabilities) — see §18.

11 Aggregator

Aggregation happens in two tiers:

1. **Scan tier** — `sub_11af80` (scan-result handler, 19 callees) is the **sole caller** of the classifier `sub_102d48`. All memory-scan findings are normalised into named records here before they reach the report layer.
2. **Record tier** — emitted records are chained onto the global list at `0x291960` (allocated lazily in `sub_0376e4` / `sub_0371e0`), then serialised by the report builder for transport.

Finding 11 (Aggregation choke-point). The scan tier has a **fan-in = 1** spine (`sub_11af80` → `sub_102d48`). It is efficient but a single logical failure (precondition false, exception, or the classifier returning early) drops *all* integrity events of that batch at once rather than degrading gracefully. **CRITICAL** · **HIGH**.

12 Reporting Pipeline

12.1 Channel registration `sub_0371e0` **CONFIRMED**

All telemetry rides one logical channel, `COREREPORT` (interned key id `0xc2ee` via `sub_0f8948`, fan-in 14). Registration binds `tdm_report` and builds the record skeleton with fields tagged `TSS`:

```
0x0371e0 report_register:
    ldrb w8,[0x29bc39]; tbz → first attempt path
    bl sub_0a231c("tdm_report",1,1,1) ; strb w0,[0x29bc38] ; AVAILABLE = result
    cbz w0, fail ; register failed
    ...
    "COREREPORT" open ; build fields via sub_017518 ; event ids 0x1ae14/0x18704/0x9c44
fail:
    ldr w9,[0x29bc3c]; add; str; cmp w9,#3
    b.lt return
    strb wzr,[0x29bc38] ; ← after 3 failures: mark channel PERMANENTLY unavailable
```

12.2 Event emit `sub_0373e8` — five silent drops **CONFIRMED**

```
0x0373e8 report_emit( x0=ctx, x1=msg ):
    cbz x0, ret ; (1) no context → DROP (returns, no emit)
    ldr x20,[x0,#8]; cbz x20, ret ; (2) no inner handle → DROP
    bl sub_0f8948(0xc2ee) ; COREREPORT key
    ldr x8,[x20]; ldr x8,[x8,#0x20]; blr x8
    cbz x0, ret ; (3) channel lookup null → DROP
    ldr x8,[x0]; ldr x8,[x8,#0x40]; adr x1,"COREREPORT"; blr x8
    cbz x0, ret ; (4) COREREPORT open null → DROP
    ldr x8,[x0]; ldr x2,[x8,#0x28]; mov x1,msg; br x2 ; (5) emit (tail)
```

Finding 12 (Reporting is fail-open and centralised). Emission is guarded by four null-checks that each **return** with **no error and no retry**; and the whole channel can be switched off by a single byte at **0x29bc38** (set to 0 on init failure, §12.1, or by any stray write). A detection that runs perfectly still produces **zero server-side evidence** if the channel object is not up. This is the most consequential structural weakness in the module. **CRITICAL** · **CONFIRMED**. Remediation: H-1, H-2, H-6.

13 Transport

13.1 Dual-path send/recv with raw-syscall gateway **CONFIRMED**

Network I/O has two paths selected by the transport bitfield at **0x2ab430**: the libc symbol, or a **raw syscall** through **sub_0a1414** (fan-in 9), which invokes **syscall(0x85=sendto)** / **syscall(0x1d=recvfrom)** to defeat libc-symbol hooking.

```
0x0f8210 net_send:
    ldr w8,[0x2ab430]; tbnz w8,#13 → syscall path
    cbnz w4 → syscall path
    bl _send ; else libc send
syscall path:
    ... mov w0,#0x85 ; bl sub_0a1414 ; raw sendto via syscall()
    ; post-send callback @0x2ab448 if set
0x0f829c net_recv: symmetric (bit14, syscall 0x1d=recvfrom, callback @0x2ab450)
```

13.2 Higher-level transport

NSURLSession / **NSURLConnection** / **NSMutableURLRequest** provide HTTP(S) upload of serialised reports; **getaddrinfo** / **socket** / **connect** / **poll** / **select** provide the low-level path. Proxy state is inspected (§6.2), giving the server a signal to distrust intercepted sessions.

Strength. The raw-syscall gateway and proxy probe are good anti-tamper choices: they make client-side interception harder and give the server MITM signals. The residual risk is **upstream** — if evidence is dropped before transport (§11, §12), a robust transport carries nothing.

14 Call-Graph Analysis

Static graph: 10,121 functions, 36,348 internal edges, 7,647 resolved import calls (289/289 stubs mapped via the indirect symbol table). Fan-in/out below are **lower bounds** for CFF/VM functions (§2.2, §10.1).

14.1 Security-relevant central nodes (by role)

Function	Role	Fan-in	Fan-out	Why it matters
sub_017518	Telemetry field constructor	187	0	Every report field is built here
sub_0f8948	COREREPORT key intern	14	2	Both register and every emit depend on it
sub_0a1414	Raw-syscall gateway	9	1	Transport + anti-hook syscalls funnel here
sub_1f32b4	Region → file mapper (injection)	5	2	Multiple injection detectors depend on it
sub_148b48	VM region read/verify/repair	5	5	Shared code-integrity primitive
sub_0373e8	Report emit	5	6	All emits pass here (5 silent drops)
sub_0371e0	Channel register	5	5	Owens the availability flag
sub_1f2ea4	dyld image enumerator	4	8	Shared by injection detectors
sub_102d48	Scan-event classifier	1	7	Sole spine for all ms_/mrpcs_events

High fan-in leaf utilities (`sub_101184` :364, `sub_0176ec` :202, `sub_017470` :88...) are allocator/log/stack-chk helpers — high centrality but low *security* specificity, so excluded from the security ranking.

14.2 Interpretation

The security graph is **hub-and-spoke around reporting**: detectors (many, redundant) converge onto a **narrow evidence funnel** (`sub_102d48` → `0x291960` → `sub_0371e0/sub_0373e8` → `transport`). Detection breadth is good; evidence-path redundancy is the deficiency (§18, §23).

15 Return-Value Analysis

15.1 Debugger check `sub_037004` — proven semantics CONFIRMED

```
bl    _sysctl          ; fill kinfo_proc
cmn   w0, #1 ; cset w8, ne ; w8 = (sysctl_ret != -1) → "call succeeded"
ldr   w9, [sp, #0x40]   ; kp_proc.p_flag
and   w9, w9, #0x800    ; isolate P_TRACED
and   w0, w8, w9, lsr #11 ; RETURN = success AND traced_bit
```

Return	Meaning
1	sysctl succeeded and P_TRACED set → debugger present
0	either not traced, OR sysctl failed → reported identically as "clean"

So `0` = *clean/not-detected* and `1` = *detected*, but `0` conflates "no debugger" with "could not check". This is the canonical fail-open (§16). CONFIRMED from the AND with `w8`.

15.2 Init return CONFIRMED

Per §4.1: `AnoSDKInit` returns a boolean of *input sign*; `sub_02d69c` returns constant `0`. The value carries no init-success information.

15.3 Report emit / register returns CONFIRMED

`sub_0373e8` returns without a distinct success/failure code on its drop paths (the caller cannot distinguish "emitted" from "dropped"). `sub_0371e0` returns null on failure and self-disables after 3 tries. In both, **callers do not receive an actionable status** — the pattern that enables Event-Drop (§16).

15.4 Method — how return semantics were established

For each critical function the branch that consumes the return (`CMP/TST` then `B.EQ/B.NE/CBZ/CBNZ`) was read directly; meanings were **not** inferred from names. Where the consumer is a C++ virtual call or executes in the VM, the semantic is marked INFERENCE rather than CONFIRMED.

16 Fail-Open Analysis

A construct is "fail-open" when an error, null, or unexpected value causes the system to treat the state as *clean* or to *discard* a detection, instead of escalating to "suspicious / retry / server-verify".

Site	Trigger	Wrong-safe behaviour	Class	Conf.
<code>sub_037004</code>	<code>sysctl</code> returns <code>-1</code> (blocked/ hooked/denied)	Returns <code>0</code> = "not debugged"	Detection Fail-Open	CONFIRMED
<code>sub_0f0fd0</code> / <code>sub_02d69c</code>	Any init failure	Init still returns "ok"; backend returns <code>0</code>	Validation Failure	CONFIRMED
<code>sub_0376e4</code> (A/B/C)		Subsystem silently skipped		CONFIRMED

	Subsystem/field/monitor construct returns 0		Detector Blindness	
sub_0373e8 (×5)	Any null in ctx/handle/channel	Event dropped, no error	Event Drop	CONFIRMED
sub_0371e0 (0x29bc3c)	3 failed registrations	Channel disabled permanently	Reporting Failure	CONFIRMED
sub_102d48 (sub_10298c)	Precondition false	Whole batch of scan events skipped	Aggregation Failure	HIGH
0x29bc38 write	Flag cleared for any reason	All telemetry silenced	Central Fail-Open	CONFIRMED

Systemic pattern. The module consistently chooses `error → clean/drop` over `error → suspicious/retry/verify`. Individually each is a defensible micro-optimisation (don't crash the game); collectively they mean **the absence of evidence is silently produced by many independent faults**. The server therefore cannot distinguish "clean device" from "detection pipeline degraded". Fixing this is the central theme of §20.

17 Critical Security Offsets

Full template per the request. All offsets are VA (= file offset within `__TEXT`).

C-1 · Report channel emit

Offset	0x0f373e8 (sub_0373e8)
Security role	Sole event-emit path into the <code>COREREPORT</code> channel
Category	Reporting / Dispatcher
Why critical	Five <code>cbz...ret</code> guards drop the event with no error; every detector's evidence passes here
Callers / Callees	5 callers (router, scan handler, detectors) / sub_0f8948 + 3 virtual calls
Return usage	No distinct success/fail code returned to caller on drop paths
Failure scenario	Channel object not yet built (early detection, or after §12.1 disable) → event silently lost
Consequence	Confirmed cheat produces zero server evidence
Evidence	§12.2 disassembly · CONFIRMED
Defensive fix	Queue-and-retry with a bounded durable buffer; emit a distinct "telemetry_unavailable" heartbeat so the server sees the gap (H-1, H-6)

C-2 · Report availability flag

Offset	0x29bc38 (data) · owner sub_0371e0
Security role	Master on/off for all telemetry
Category	Reporting / Security-context
Why critical	One unauthenticated byte silences the whole evidence pipeline; auto-cleared after 3 failures
Failure scenario	Registration flaps 3× (transient) → permanently off for the session
Consequence	Process-wide telemetry blackout
Evidence	§5, §12.1 · CONFIRMED
Defensive fix	Replace "disable after N" with exponential-backoff re-arm; treat "off" as a reportable anomaly, not a steady state (H-2)

C-3 · Scan-event classifier spine

Offset	0x102d48 (sub_102d48), sole caller sub_11af80
--------	---

Security role	Converts every memory-scan finding into a named event
Category	Aggregator / Integrity
Why critical	fan-in = 1; a precondition-false or early return drops an entire scan batch
Failure scenario	<code>sub_10298c</code> gate false → no <code>ms_/mrpcs_</code> events for that cycle
Consequence	Integrity/inline-hook blindness indistinguishable from "clean"
Evidence	§9.2, §11 · HIGH
Defensive fix	Redundant secondary classifier + emit an explicit "scan_incomplete" event on the skip path (H-4)

C-4 · Debugger check fail-open

Offset	0x37004 (<code>sub_037004</code>)
Security role	Primary <code>P_TRACED</code> anti-debug
Category	Detector / Debugger
Why critical	Result ANDed with <code>sysctl</code> success → a blocked syscall reads as "not debugged"
Return usage	<code>0=clean</code> conflates "no debugger" and "check failed"
Failure scenario	<code>sysctl</code> denied/hooked → 0 → clean
Consequence	Anti-debug defeated by suppressing one call, invisibly
Evidence	§15.1 · CONFIRMED
Defensive fix	Distinguish failure from clean: on <code>sysctl</code> error emit "debug_check_unavailable" (suspicious), corroborate with <code>task_info</code> /timing (H-3)

C-5 · Init success not observable

Offset	0xf0fd0 / 0x2d69c
Security role	Bring-up entry & backend
Category	Init / Validation
Why critical	Return derived from input; backend constant 0 → host can't verify protection came up
Consequence	Partial/failed bring-up runs unnoticed (with §4.3 silent skips)
Evidence	§4.1 · CONFIRMED
Defensive fix	Return a real init-status bitmap; server refuses sessions whose attestation shows missing subsystems (H-5)

18 Single Points of Failure

#	Node	Depends-on-it	If it fails (unintentionally)	Sev.
S1	0x29bc38 flag / <code>sub_0373e8</code>	Every detector's evidence	Total telemetry loss, silent	CRITICAL
S2	<code>sub_102d48</code> (fan-in 1)	All <code>ms_/mrpcs_</code> integrity events	Integrity-scan blindness	CRITICAL
S3	<code>sub_0f8948</code> (key intern)	Register + every emit (fan-in 14)	Reporting cannot address the channel	HIGH
S4	<code>sub_017518</code> (field ctor)	All report fields (fan-in 187)	Records built empty/garbled	HIGH
S5	<code>sub_0a1414</code> (syscall gw)	Anti-hook transport + syscalls (fan-in 9)	Upload path / covert syscalls lost	HIGH
S6	<code>sub_0376a0</code> / table 0x290580	All subsystem starts / ioctl dispatch	Subsystems never start / cmds misroute	HIGH
S7	<code>sub_1f32b4</code> · <code>sub_1f2ea4</code> · <code>sub_148b48</code>	Multiple injection/integrity detectors	Whole detector class blinded	MEDIUM

S8	0x291960 list head	Aggregated record chain	Records orphaned before serialisation	MEDIUM
----	--------------------	-------------------------	---------------------------------------	---------------

Note the asymmetry: **detection** is redundant (many detectors), but **evidence handling** (S1–S4, S8) is not. The protection's resilience is bounded by its *least* redundant stage — reporting.

19 Attack-Surface Matrix (Defensive)

"Weakness" = an *unintentional* failure mode a developer must harden; not an exploitation recipe.

Offset	Component	Weakness	Failure effect	Sev.	Evidence	Defensive fix
0x29bc38 / 0xf373e8	Report channel	Central fail-open + silent drops	Telemetry blackout	CRIT	\$12	H-1,H-2,H-6
0x102d48	Scan classifier	fan-in 1 spine, skippable	Integrity blindness	CRIT	\$9.2,\$11	H-4
0x37004	Anti-debug	error → clean fail-open	Debugger undetected	HIGH	\$15.1	H-3
0xf0fd0 / 0x2d69c	Init	Return not tied to success	Failed bring-up unnoticed	HIGH	\$4.1	H-5
0x376e4	Router	3× silent subsystem skip	Missing detector, no signal	HIGH	\$4.3	H-5,H-4
0x29bc3c	Report register	Disable-after-3	Permanent session blackout	HIGH	\$12.1	H-2
0x290580 / 0x26398	loctl dispatch	Opaque cmd table (audit gap)	Possible unhandled-cmd default	MED	\$10.1	H-8
0x2ab430	Transport mode	Unauthenticated toggle	Forced onto weaker path	MED	\$13.1	H-7
0x291960	Record list	Single global chain	Orphaned evidence	MED	\$11	H-6
1f32b4/1f2ea4/148b48	Shared detector deps	Multi-detector coupling	Class-wide blindness	MED	\$7.2,\$8.1	H-4

20 Defensive Hardening Plan

H-1 · Make the evidence path fail-closed

Replace each `cbz...ret` drop in `sub_0373e8` with enqueue-to-durable-buffer + bounded retry. If the channel is truly unavailable, persist the event (encrypted, monotonic-counter tagged) and flush later. **An event must never vanish because a pointer was null.**

H-2 · Remove the single-flag kill switch

`0x29bc38` "disable after 3 failures" should become exponential-backoff re-arm. The channel being down is itself a high-priority event; surface it, don't latch it off.

H-3 · Fail-closed detectors (error ≠ clean)

In `sub_037004` and peers, split the result into `{clean, detected, check-failed}`. On `check-failed` emit a "suspicious/verify" event and corroborate via an independent method (`task_info` exception-port state, timing, raw-syscall variant). One blocked `sysctl` must not read as clean.

H-4 · Redundant, self-announcing integrity path

Give `sub_102d48` a second, independently-scheduled classifier; on the `sub_10298c` skip path emit `scan_incomplete`. Duplicate the shared detector primitives (`sub_1f32b4` / `1f2ea4` / `148b48`) or cross-validate their outputs so losing one does not blind a whole class.

H-5 · Observable init & subsystem attestation

`AnoSDKInit` should return a real status bitmap of the ~59 initializers/7 subsystems; the router's silent `cbz` skips (§4.3) should record which subsystem failed. The server refuses/segregates sessions whose attestation shows missing detectors.

H-6 · Signed, ordered, replay-proof telemetry

Chain records with a monotonic counter and per-session key (keychain id from §6.1); the server detects **gaps** (dropped/suppressed events) and **replays**. This converts today's silent drops into server-visible anomalies.

H-7 · Protect the security context

Shadow-copy and checksum the `0x29bxxx` / `0x2ab430` globals; verify before use. A single stray/hostile write to a flag should be detectable, not authoritative.

H-8 · Audit the opaque dispatcher

With source or symbolic execution, enumerate every `cmd` handled by `sub_026398` /table `0x290580` and prove there is no benign default-fallthrough for unknown/malformed commands (fail-closed default).

H-9 · Server-side authority for verdicts

Any ban/kick decision must be corroborated server-side from the signed event stream; never trust a single client boolean. Client detectors become *evidence sources*, not *judges*.

Priority order: H-1 & H-2 (stop silent evidence loss) → H-3 (fail-closed detectors) → H-6 (signed gap-detectable stream) → H-4/H-5 (redundancy + attestation) → H-7/H-8/H-9 (context, dispatcher, server authority).

23 Failure Propagation Map

- (1) Debugger check `sub_037004` : `sysctl` blocked → returns 0 "clean"
 ↳ detection event never created → server sees "no debugger" (false-clean)
- (2) Subsystem init `sub_0376e4` : `construct` returns 0 → `cbz` silent skip
 ↳ detector never starts → its whole event class absent → server unaware
- (3) Scan spine `sub_102d48` : precondition false / early return
 ↳ `ms_/mrpcs_` batch dropped → CRC + inline-hook events lost → integrity "clean"
- (4) Report channel `0x29bc38` cleared OR `sub_0373e8` null-guard
 ↳ EVERY event dropped, all detectors → total blackout
 ▼
 server receives incomplete/empty security state → cannot distinguish
 "clean device" from "degraded/suppressed client" → verdict starved of evidence

The four independent faults above all converge on the **same observable**: an empty or partial report that the server currently reads as benign. This convergence is why §20 prioritises making *absence of evidence* itself a server-visible, signed signal (H-1/H-2/H-6).

24 Top 10 Security Weak Points

Risk = DetectorDependency + CallGraphCentrality + ReportingDependency + Fail-OpenProbability + IntegrityImpact (each 0–5; max 25).

Rank	Offset	Function / node	Security layer	Why critical (harden this)	Failure consequence	Score	Sev.	Conf.
1	<code>0x29bc38</code> · <code>0xf373e8</code>	Report channel gate + emit	Reporting	One flag / 5 null-guards silence all evidence	Total telemetry blackout	24	CRIT	CONF
2	<code>0x102d48</code>	Scan-event classifier	Aggregator/ Integrity	fan-in 1 spine for all <code>ms_/_mrpcs_</code>	Integrity blindness	20	CRIT	HIGH
3	<code>0x0a1414</code>	Raw-syscall gateway	Transport/ Core	9 callers; anti-hook I/O funnel	Upload + covert syscalls lost	19	HIGH	CONF
4	<code>0xf0fd0</code> · <code>0x2d69c</code>	Init entry + backend	Init	Success not observable to host/server	Failed bring-up unnoticed	17	HIGH	CONF
5	<code>0x0371e0</code> (<code>0x29bc3c</code>)	Channel register / disable-after-3	Reporting	Transient flaps latch reporting off	Permanent session blackout	17	HIGH	CONF
6	<code>0x37004</code>	Anti-debug (P_TRACED)	Detector	error – clean fail-open	Debugger undetected	15	HIGH	CONF
7	<code>0x376e4</code>	Router subsystem dispatch	Dispatcher	3× silent subsystem skip	Missing detector, no signal	15	HIGH	CONF
8	<code>0x017518</code>	Telemetry field constructor	Core	187 callers; every field	Records built empty/garbled	14	HIGH	CONF
9	<code>0x1f32b4</code> · <code>0x148b48</code>	Shared injection/ integrity deps	Detector/ Integrity	Multi-detector coupling (5+5)	Whole detector class blinded	13	MED	CONF

10	0x26398 / 0x290580	loctl CFF dispatcher	Dispatcher	Opaque cmd table — audit gap	Possible unhandled-cmd default	12	MED	CONF*
----	--------------------	----------------------	------------	------------------------------	--------------------------------	----	-----	-------

CONF* = structure confirmed; exact per-command behaviour requires source/symbolic execution (§10.1). These are the ten places a developer should harden so the system does not lose the ability to *detect or report* cheating — deliberately not a list of how to disable anything.

25 Final Technical Conclusions

- 1. **Identity & shape.** A stripped ARM64 iOS dylib implementing the ANOGS/AnoSDK anti-cheat: 19 `AnoSDK*` exports, 59 initializers, ~10k functions, a custom protection VM, and control-flow-flattened dispatch. Anti-*tamper* engineering is strong. **CONFIRMED** (identity **INFERENCE**).
- 2. **The weakness is logical, not cryptographic.** Detection is broad and redundant, but the detect→event→aggregate→report chain narrows to a few unprotected, fail-open nodes. The module repeatedly prefers `error → clean/drop` over `error → suspicious/verify`.
- 3. **Four confirmed silent-failure classes** (detection fail-open `0x37004`; silent subsystem skip `0x376e4`; event drop `0x0373e8`; channel self-disable `0x29bc3c/38`) all converge on one observable — an empty report the server treats as benign.
- 4. **Init/verdict trust.** `AnoSDKInit` returns an input-derived boolean, not a health status; neither host nor server can currently tell a fully-armed client from a degraded one.
- 5. **Highest-leverage fixes:** make the evidence path fail-closed and signed (H-1,H-2,H-6), make detectors distinguish "can't check" from "clean" (H-3), and give init/subsystems real attestation (H-4,H-5). These change silent losses into server-visible anomalies without touching the (already solid) obfuscation.
- 6. **Method integrity.** Every offset here is a VA identical to its file offset (§2.2) and was taken from disassembly with dual-disassembler and indirect-symtab cross-checks; semantics were read from the consuming branch, not guessed from names. Findings are labelled CONFIRMED / HIGH / INFERENCE so the maintainer can re-verify each independently.

Scope statement. This audit is strictly defensive. It documents where the anti-cheat may *lose* detection or evidence and how to harden those points. It intentionally contains no patch bytes, NOP/RET edits, hook code, or bypass procedure, and none should be inferred from it.

Appendix A · Re-verification quick-reference

Claim	How to reproduce
Offset model VA=FileOff	Compare LC_SEGMENT_64 <code>vmaddr</code> vs <code>fileoff</code> for <code>__TEXT/ __DATA</code> ($\Delta=0$)
Stub → import names	Indirect symbol table (LC_DYSYMTAB <code>indirectsymoff</code>) indexed by each section's <code>reserved1</code>
sub_037004 fail-open	Disassemble 0x37004; read <code>and w0,w8,w9,lsl#11</code> (w8 = sysctl success)
sub_0373e8 drops	Disassemble 0xf373e8; count the 4 <code>cbz...ret</code> + tail <code>br x2</code>
Router gate	Disassemble 0x376a0/0x376e4; table 0x29bc10, byte-jumptable 0x23c6d0
CFF dispatcher	Disassemble 0x26398; successor table 0x290580 (8-byte entries into <code>__text</code>)

— End of report — Defensive Anti-Cheat Security Audit · anogs45orgnll · generated 2026-08-26 —